Jupyter Notebook on Android

Complete Termux Installation, Error Prevention & Troubleshooting Guide

Step-by-step setup • Preventive maintenance • Real-world error solutions

```
pkg update && pkg upgrade -y
export ANDROID_API_LEVEL=21
pkg install python-psutil -y
pip install -upgrade pip setuptools wheel
pip install jupyter
jupyter notebook -no-browser -ip=127.0.0.1
```

Edition: Updated with Real-World Fixes
Date: August 24, 2026
Scope: Termux (Android) + Python 3 + Jupyter Notebook
Includes: Error Prevention + Solutions

This guide includes preventive steps to avoid common errors before they occur. Updated based on real installation feedback.

Contents

1 Introduction

1.1 What Is Termux?

Termux is a free, open-source terminal emulator and Linux environment app for Android. It does not require root access. It ships its own minimal package ecosystem (based on Debian's `apt`, wrapped by a friendlier front-end called `pkg`) so you can install real command-line tools – Python, Git, Node.js, OpenSSH, compilers, and hundreds of other packages – directly on your phone or tablet, inside a sandboxed, per-app storage area.

1.2 What Is Jupyter Notebook?

Jupyter Notebook is a web-based interactive environment for writing and running code (mainly Python) in cells, mixed with formatted text, equations, and visual output. Even though it is "web-based," the server runs locally – in this case, inside Termux on your phone – and you simply open the interface in a normal mobile browser tab.

1.3 Why Run Jupyter on Android?

- Practice or prototype Python code anywhere, without a laptop.
- Analyse small CSV files, test snippets, or revise for exams on the go.
- Zero cost, fully offline once installed (no cloud dependency).
- Useful as a backup environment when your primary machine is unavailable.

1.4 Prerequisites & System Requirements

- **Android version:** 7.0 (Nougat) or newer. Android 11+ is recommended for full compatibility with the latest Termux builds and storage permissions.
- **Storage:** At least 2–3 GB of free internal storage (Jupyter, Python, and the compiler toolchain together take up real space).
- **RAM:** 3 GB+ is comfortable; it will still run on less, just slower, and large packages (NumPy, Pandas) may struggle to compile from source on very low-RAM devices.
- **Internet connection:** required for the initial package downloads.
- **A mobile browser:** Chrome, Firefox, or any modern browser, to open the Jupyter interface.

2 Step-By-Step Installation

2.1 Step 1 – Install Termux the Right Way

Do **not** install Termux from the Google Play Store. That build has been deprecated by the Termux team for years – it is missing recent bug fixes, security patches, and features, and can silently attempt to overwrite a working GitHub installation. Always use the official **GitHub releases** page.

1. Install **Termux** from within it.
   download the APK directly from https://github.com/termux/termux-app/releases (grab

the `universal` APK unless you know your device's exact CPU architecture).

2. If your browser or file manager asks for permission to "install unknown apps," allow it – this is normal for GitHub installs and is what avoids the deprecated Play Store build.
3. If you plan to use `termux-api` commands later (Section 5.4), also install the **Termux:API** add-on app from the *same source* (GitHub) as Termux itself.

Why the source matters

Termux and its add-on apps must share the same package signature to talk to each other. Mixing a Play Store Termux with an F-Droid add-on (or vice versa) breaks inter-app communication with a signature mismatch.

2.2 Step 2 – First Launch: Update the Package Index

Open Termux and run:

```
pkg update -y
pkg upgrade -y
```

`pkg update` refreshes the list of available packages and their versions from Termux's repositories. `pkg upgrade` then installs newer versions of anything already on your system. Always do this first on a fresh install, and periodically afterwards.

2.3 Step 3 – Grant Storage Access

```
termux-setup-storage
```

This creates a `~/storage` folder inside Termux, symlinked to your phone's shared storage (Downloads, DCIM, Music, etc.), after you accept the Android permission prompt. You need this if you want Jupyter notebooks to read or save files outside Termux's private sandbox (e.g. a CSV from your Downloads folder).

2.4 Step 4 – Install Python

```
pkg install python -y
```

This installs Python 3 together with `pip`, precompiled specifically for Android's Bionic C library. Verify it:

```
python --version
pip --version
```

2.5 Step 5 – Install Build & Native Dependencies

Several of Jupyter's dependencies (most notably `pyzmq`, which handles the notebook's messaging layer) need to compile native code. Install the toolchain up front to avoid failures later:

```
pkg install -y build-essential clang libzmq rust cmake pkg-config
```

- `build-essential` – meta-package pulling in `make`, `binutils`, and related build tools.
- `clang` – the C/C++ compiler Termux uses (there is no `gcc` on Termux; `clang` is the standard).
- `libzmq` – the native ZeroMQ messaging library that `pyzmq` links against.
- `rust` – needed by some build backends (e.g. `maturin`) used by newer versions of `pyzmq` and by packages such as `tokenizers`.
- `cmake`, `pkg-config` – general build-system helpers many Python C-extensions rely on.

Installing this toolchain before running `pip install jupyter` prevents most of the errors described in Section 4.

2.6 Step 6 – Upgrade pip, setuptools, and wheel

```
pip install --upgrade pip setuptools wheel
```

An outdated `pip` is a common, avoidable source of confusing build failures.

2.7 Step 6a – (NEW - PREVENTIVE) Set Android API Level

This step prevents the "maturin failed" error when building Rust-based packages like `rpds-py`. Run this command:

```
export ANDROID_API_LEVEL=21
```

Sets the Android API level to 21 (Android 5.0), which tells Rust and maturin (the build tool) which Android version to compile for. This is required before installing packages with Rust components.

To make this permanent (so it's always set in future sessions), add it to your `.bashrc`:

```
echo 'export ANDROID\_API\_LEVEL=21' >> ~/.bashrc
echo 'export LDFLAGS="-lm -lcompiler_rt"' >> ~/.bashrc
source ~/.bashrc
```

After running these commands, `export ANDROID_API_LEVEL=21` will be set automatically every time you open Termux!

2.8 Step 6b – (NEW - PREVENTIVE) Install Pre-compiled System Packages

Instead of letting `pip` compile heavy packages from source (which fails or takes hours), install Termux's pre-compiled versions first:

```
pkg install -y python-psutil python-numpy python-scipy python-matplotlib
```

- `python-psutil` – System monitoring library (prevents "platform android is not supported" error).
- `python-numpy` – Numerical computing (much faster than compiling from source).
- `python-scipy` – Scientific computing (avoids 30+ minute compilation).
- `python-matplotlib` – Plotting library (precompiled, no build time).

Without these precompiled packages, `pip install jupyter` will try to compile `psutil` and fail with "platform android is not supported." By installing precompiled versions first, this error never occurs.

This step takes 2–3 minutes for all packages. Much faster than compiling from source (which would take 30+ minutes and likely fail).

2.9 Step 7 – Clear pip Cache

Clear any previous failed package downloads to prevent re-using corrupted wheels:

```
pip cache purge
```

2.10 Step 8 – Install Jupyter

```
pip install jupyter
```

This pulls in the classic Notebook interface along with `ipykernel`, `pyzmq`, `tornado`, and the other required pieces. Because you've already set the API level and installed precompiled packages, this should now complete without errors.

Prefer the classic `jupyter notebook` interface over JupyterLab on Termux if you want the smoothest install. JupyterLab (`pip install jupyterlab`) is heavier, pulls in more native/Rust-based build dependencies, and is more prone to the build errors. It works, but expect a few extra troubleshooting steps – see Error 3.

2.11 Step 9 – Verify the Installation

```
jupyter --version
```

This should print version numbers for `jupyter core`, `notebook`, `ipykernel`, and related packages – confirming a working install.

2.12 Step 10 – Launch Jupyter Notebook

```
jupyter notebook --no-browser --ip=127.0.0.1 --port=8888
```

- `--no-browser` – stops Jupyter from trying to auto-launch a system browser, which Termux does not reliably have registered.
- `--ip=127.0.0.1` – binds the server to the loopback interface only, so it is reachable only from the same device (safer default).
- `--port=8888` – the port to listen on; change it if 8888 is already taken.

2.13 Step 11 – Open the Notebook in Your Browser

After Step 10, Termux prints a URL similar to:

```
http://127.0.0.1:8888/?token=6f1a9c2e4b...
```

Long-press to copy the full URL (including the token), switch to Chrome/Firefox on the same phone, and paste it into the address bar. The Jupyter dashboard should load, and you can create a

new Python 3 notebook from there.

If the page loads as a blank/black screen, this is usually a browser cache issue, not a server problem. Try:

1. Hard refresh: `Ctrl+Shift+R` on desktop browsers, or hold the refresh button and select "Clear cache and reload" on mobile.
2. Clear browser cache: Settings → Privacy/History → Clear cached images and files.
3. Try a different browser: Chrome, Firefox, or Edge.

2.14 Step 12 – Install Data-Science Packages the Smart Way

For heavy scientific packages, prefer Termux's own precompiled binaries over building from source with `pip`:

```
pkg install -y python-pandas
```

Only fall back to `pip install X` for a package that Termux does not package – see Error 8 in Section 4 for why this matters.

2.15 Step 13 – Keep Jupyter Running in the Background

Android aggressively kills background processes to save battery. Two things make a long-running Jupyter session reliable:

```
pkg install tmux -y
tmux new -s jupyter
jupyter notebook --no-browser --ip=127.0.0.1 --port=8888
# to detach and leave it running: press Ctrl+b, then d
termux-wake-lock
```

`tmux` keeps the Jupyter process alive even if you close the Termux window or switch apps; `termux-wake-lock` asks Android not to suspend the CPU. Reattach any time with `tmux attach -t jupyter`. Also see the battery-optimisation note in Error 7.

2.16 Step 14 – Create a Quick-Launch Alias

Add a shortcut so you don't have to retype the full command every time:

```
echo 'alias jn="jupyter notebook --no-browser --ip=127.0.0.1 --port=8888"' >> ~/.
    bashrc
source ~/.bashrc
```

From now on, just type `jn` to start Jupyter.

3 Error Prevention Checklist

Before you encounter errors, verify these preventive steps are complete:

Preventive Step 1: Have you run `pkg update` and `pkg upgrade`? (Step 2)

Preventive Step 2: Have you installed `build-essential`, `clang`, `libzmq`, and `rust`? (Step 5)

Preventive Step 3: Have you set `export ANDROID_API_LEVEL=21`? (Step 6a)
Check: `echo $ANDROID_API_LEVEL` should print `21`

Preventive Step 4: Have you installed `python-psutil` and `python-numpy`? (Step 6b)
Check: `pip list | grep psutil` should show a result

Preventive Step 5: Have you cleared pip cache with `pip cache purge`? (Step 7)

Preventive Step 6: Does `jupyter -version` work? (Step 9)

If all six preventive steps are complete, you have eliminated 99% of common Jupyter installation errors before they occur!

4 Known Errors and Their Fixes

This section covers errors that might still occur, even with preventive steps. Each entry shows the symptom, the underlying cause, and the exact fix.

4.1 Error 1 – *Failed building wheel for psutil*

```
ERROR: Failed building wheel for psutil
Building wheel for psutil (pyproject.toml) ... error
error: command '.../aarch64-linux-android-clang++' failed with exit code 1
```

Cause

You skipped Step 6b. The system tried to compile `psutil` from source, but Android isn't officially supported by psutil's build system.

```
pkg install python-psutil -y
pip cache purge
pip install jupyter
```

4.2 Error 2 – *ImportError When Starting Jupyter (pyzmq linking)*

```
ImportError: dlopen failed: cannot locate symbol referenced by
".../zmq/backend/cython/_zmq.cpython-312-...so"
```

Cause

The ZeroMQ Cython extension compiled successfully, but the resulting shared object was not explicitly linked against Termux's `libpython`. Android's linker is stricter than a typical desktop Linux linker and refuses to resolve the missing symbol at runtime.

```
pkg install patchelf -y
python --version
patchelf --add-needed libpython3.12.so \
  $PREFIX/lib/python3.12/site-packages/zmq/backend/cython/_zmq.cpython-312-*.so
```

4.3 Error 3 – *maturin failed: Android API level not found*

```
 maturin failed
Caused by: Failed to determine Android API level.
Please set the ANDROID_API_LEVEL environment variable.
```

Cause

You skipped Step 6a. Rust-based packages (like `rpds-py`) require the Android API level to be set before compilation.

```
export ANDROID_API_LEVEL=21
export LDFLAGS="-lm -lcompiler_rt"
pip cache purge
pip install jupyter
```

4.4 Error 4 – `termux-setup-storage` Does Nothing / Permission Denied

```
ls: /data/data/com.termux/files/home/storage/shared: Permission denied
```

Cause

The Android runtime permission prompt was dismissed, or the Termux app does not have the "Files and media" permission enabled (common on Android 11+ with scoped storage).

```
termux-setup-storage
# If no prompt appears, enable it manually:
# Android Settings > Apps > Termux > Permissions > Files and media > Allow
```

4.5 Error 5 – `jupyter: command not found`

```
bash: jupyter: command not found
```

Cause

`pip` installed the `jupyter` launcher script under `~/.local/bin`, which is not on your shell's `PATH` by default in some configurations.

```
echo 'export PATH=$PATH:$HOME/.local/bin' >> ~/.bashrc
source ~/.bashrc
jupyter --version
```

4.6 Error 6 – `OSError: [Errno 99] Cannot assign requested address`

```
OSError: [Errno 99] Cannot assign requested address
```

Cause

Jupyter was told to bind to a specific IP address (often a Wi-Fi IP that has since changed, or an interface some ROMs restrict) that is not currently available on the device.

```
# Safe default: bind to loopback only
jupyter notebook --ip=127.0.0.1 --port=8888 --no-browser

# For LAN access from a laptop on the same Wi-Fi (optional):
pkg install net-tools -y
ifconfig            # find your phone's local IP
jupyter notebook --ip=0.0.0.0 --port=8888 --no-browser
```

4.7 Error 7 – Jupyter Stops When You Lock the Phone or Switch Apps

Cause

Android kills background processes, including Termux sessions, to save battery – especially if the Termux notification is dismissed or battery optimisation is enabled for the app.

```
pkg install tmux -y
tmux new -s jupyter
jn   # or: jupyter notebook --no-browser --ip=127.0.0.1
termux-wake-lock
# Then: Android Settings > Battery > Termux > Unrestricted / No restrictions
```

4.8 Error 8 – Blank/Black Screen in Browser

```
URL opens but page shows only black screen
```

Cause

Browser hasn't loaded JavaScript assets, usually due to cache issues or slow initial asset download. Jupyter backend is working fine.

```
# Try hard refresh first (browser-specific):
# - Desktop: Ctrl+Shift+R
# - Mobile: Hold refresh button > "Clear cache and reload"
# - All: Wait 15 seconds for assets to load

# If that doesn't work, clear browser cache:
# Settings > Privacy/History > Clear cached images and files

# Or try different port:
jupyter notebook --no-browser --ip=127.0.0.1 --port=9999
```

4.9 Error 9 – SSL Certificate Errors on pip/curl

```
SSL: CERTIFICATE_VERIFY_FAILED
```

Cause

The CA certificate bundle is missing or outdated, which can happen right after a fresh Termux install.

```
pkg install ca-certificates -y
update-ca-certificates
```

4.10 Error 10 – `setlocale` Warning

```
bash: warning: setlocale: LC_ALL: cannot change locale (en_US.UTF-8)
```

Cause

Android does not ship full desktop-style locale data, so Termux's shell cannot resolve the default locale it expects. This is cosmetic and does not affect Jupyter's functionality.

```
echo 'export LANG=en_US.UTF-8' >> ~/.bashrc
echo 'export LC_ALL=en_US.UTF-8' >> ~/.bashrc
source ~/.bashrc
```

4.11 Error 11 – No Space Left on Device

```
E: You don't have enough free space in /data/data/com.termux/files/usr/...
```

Cause

Termux's downloaded package archives (`.deb` cache) and pip's wheel cache accumulate over time and can quietly consume several hundred MB.

```
pkg clean
apt autoremove -y
pip cache purge
```

5 Termux Command Reference

Beyond installing Jupyter, it helps to know your way around Termux itself. This section is a categorised reference of commonly used commands, each with a short, plain description of what it does.

5.1 Package Management (`pkg` / `apt`)

Command	What it does
`pkg update`	Refreshes the list of available packages from Termux's repositories.
`pkg upgrade -y`	Upgrades all installed packages to their latest versions.
`pkg install <name>`	Installs a package.
`pkg uninstall <name>`	Removes an installed package.
`pkg search <term>`	Searches package names/descriptions for a keyword.
`pkg list-installed`	Lists every package currently installed.
`pkg list-all`	Lists every package available in the repositories.
`pkg show <name>`	Shows details (version, dependencies) for a package.
`pkg clean`	Clears the local package download cache to free space.
`apt autoremove -y`	Removes packages that were auto-installed as dependencies and are no longer needed.
`dpkg -l`	Lists installed packages via the lower-level `dpkg` tool.

5.2 File & Directory Management

Command	What it does
`ls -la`	Lists all files in a directory, including hidden ones, with details.
`cd <dir>`	Changes the current directory.
`pwd`	Prints the current working directory path.
`mkdir -p <dir>`	Creates a directory, including parent directories if needed.
`rm -rf <path>`	Deletes a file or directory forcefully and recursively – use with care.

Command	What it does
`cp -r <src> <dst>`	Copies a file or directory (recursively for directories).
`mv <src> <dst>`	Moves or renames a file or directory.
`touch <file>`	Creates an empty file, or updates its timestamp.
`cat <file>`	Prints a file's contents to the terminal.
`nano <file>`	Opens a file in the simple Nano text editor.
`vim <file>`	Opens a file in the Vim text editor (install with `pkg install vim`).
`find . -name "*.py"`	Searches recursively for files matching a pattern.
`du -sh <dir>`	Shows the total disk space used by a directory, in human-readable form.
`df -h`	Shows free/used storage space on all mounted filesystems.
`tar -czvf out.tar.gz <dir>`	Compresses a directory into a `.tar.gz` archive.
`unzip <file.zip>`	Extracts a `.zip` archive.
`chmod +x <script.sh>`	Marks a script as executable.

5.3 Python & pip

Command	What it does
`python --version`	Shows the installed Python version.
`pip list`	Lists all installed Python packages.
`pip show <pkg>`	Shows details about an installed package.
`pip freeze > requirements.txt`	Saves the current package list to a file.
`pip install -r requirements.txt`	Installs all packages listed in a requirements file.
`pip cache purge`	Clears pip's download cache.
`python -m venv myenv`	Creates an isolated Python virtual environment.
`source myenv/bin/activate`	Activates a virtual environment.
`deactivate`	Exits the currently active virtual environment.

5.4 Termux-Specific Utilities (`termux-api`)

These commands bridge Termux with Android system features. They require both the `termux-api` package and the companion **Termux:API** app (installed from the same source as Termux itself):

```
pkg install termux-api -y
```

Command	What it does
`termux-setup-storage`	Grants Termux access to shared Android storage (Downloads, DCIM, etc.).
`termux-open <file>`	Opens a file with the appropriate Android app.
`termux-open-url <url>`	Opens a URL in the default browser.
`termux-share <file>`	Opens the Android share sheet for a file.
`termux-clipboard-get`	Prints the current Android clipboard contents.
`termux-clipboard-set "text"`	Sets the Android clipboard contents.

Command	What it does
`termux-toast "message"`	Shows a short Android toast pop-up message.
`termux-notification --title "T"--content "C"`	Posts an Android notification.
`termux-battery-status`	Prints battery level, temperature, and charging status as JSON.
`termux-wake-lock`	Prevents Android from suspending the CPU while Termux runs in the background.
`termux-wake-unlock`	Releases the wake lock set above.
`termux-info`	Prints diagnostic information about the Termux installation.
`termux-reload-settings`	Reloads Termux's terminal preferences after editing them.

6 Tips, Best Practices & Security Notes

- **Always set Android API level BEFORE installing.** Run Step 6a before any `pip install` commands. This prevents maturin and Rust compilation errors.
- **Always install precompiled packages first.** Run Step 6b to install `python-psutil`, `python-numpy`, etc. from Termux's repository before running `pip install jupyter`. This prevents hours of failed compilation.
- **Update before installing anything heavy.** Run `pkg update && pkg upgrade -y` before installing a new large package – it prevents a lot of avoidable dependency errors.
- **Use `tmux` for anything long-running.** Jupyter, training loops, or long downloads should run inside a tmux session so they survive you switching apps.
- **Back up your work.** Termux's storage lives inside the app's private sandbox. Uninstalling the app (or a factory reset) wipes everything with no recovery. Regularly push notebooks/projects to GitHub or copy them into `~/storage/shared` (see Step 3).
- **There is no `sudo` in Termux.** You already run as the app's own user with full write access to Termux's own filesystem – `sudo` is neither installed nor needed for normal package management.
- **Keep the default `--ip=127.0.0.1` binding** unless you specifically need another device on your network to reach the notebook – it is the safer default and avoids exposing an unauthenticated port on shared Wi-Fi.
- **Treat token-based URLs as secrets.** The `?token=...` in the Jupyter URL is effectively a password; don't share screenshots of it.
- **Thermals and battery matter.** Sustained heavy computation on a phone will throttle the CPU and drain the battery quickly – for genuinely large workloads, use a laptop or a cloud notebook instead.

Appendix A – One-Shot Setup Script

The block below combines the core installation steps into one copy-pasteable sequence. Run it line by line rather than all at once the first time, so you can see and understand each step – and review Section 4 if any single line fails.

```
pkg update -y && pkg upgrade -y
termux-setup-storage
pkg install -y build-essential clang libzmq rust cmake pkg-config
export ANDROID_API_LEVEL=21
echo 'export ANDROID_API_LEVEL=21' >> ~/.bashrc
pkg install -y python-psutil python-numpy python-scipy python-matplotlib
pip install --upgrade pip setuptools wheel
pip cache purge
pip install jupyter
jupyter --version
echo 'alias jn="jupyter notebook --no-browser --ip=127.0.0.1"' >> ~/.bashrc
source ~/.bashrc
```

Appendix B – Quick Troubleshooting Checklist

1. Install fails on building psutil? → Did you run Step 6b? (`pkg install python-psutil -y`)
2. Maturin failed with Android API level? → Did you run Step 6a? (`export ANDROID_API_LEVEL=21`)
3. Jupyter starts but crashes with an `ImportError` about `_zmq`? → Error 2 (`patchelf` fix).
4. Can't read/write files outside Termux? → Error 4 (storage permission).
5. `jupyter: command not found`? → Error 5 (`PATH` fix).
6. Jupyter page loads as blank screen? → Error 8 (hard refresh or clear cache).
7. Jupyter dies when you leave the app? → Error 7 (tmux + wake-lock + battery settings).
8. Package install takes forever or fails? → Use `pkg install python-X` instead of `pip install X` for heavy packages.
9. "No space left on device"? → Error 11 (`pkg clean`, `apt autoremove`).

If you completed all 14 steps (especially Steps 6a and 6b), you should not encounter any errors. If you do, this checklist will point you to the exact fix.